TEC | Tecnológico de Costa Rica

Instituto Tecnológico de Costa Rica

Introducción a la Programación

Escuela de Ingeniería en Computadores

Primer Proyecto

Ping Pong

Documentación escrita guía

Profesor: Jeff Schmith

Estudiante:

Mónica Blanco Jiménez (2026008253)

Fecha de entrega: 18 de octubre 2026

Año: 2026

Objetivo:

Guiar al revisor de los algoritmos desarrollados por el estudiante, para asegurar la comprensión de la estructura y de las funciones principales del código mediante diagramas y una explicación breve de cada prompt relevante, para facilitar entendimiento a la hora de la defensa

Introducción

En el siguiente documento encontrará una guía de la jerarquía de archivos de este repositorio para la ejecución del archivo del proyecto.

Cabe recalcar que, aunque la documentación no fue solicitada de manera estricta en las instrucciones, se asignará solo como guía visual para la comprensión de la estructura y de algunas de las funciones más importantes de este proyecto que comparten similitudes con el archivo de tarea tkinter en el sistema de colisiones y movimiento por funciones de recursividad.

Muchas de las funciones fueron replicadas en múltiples asignaturas y las variables igual, es bastante notable por la estructura de los prompts.

1. **Funciones de Apertura**

Las funciones de apertura con forma “def función(): función.abrir_función()” son las funciones que permiten abrir con el Command en los botones mediante llamados a las funciones desde el menú, cada una tiene un comando destroy para evitar desordenes con las ventanas, estos se abren de manera inmediata para evitar que presionan el click mas de una vez y el usuario cree desordenes con las ventanas. La única que carece de esto es el propio ping pong menú en la que no se supo cómo contemplar un control más adecuado, quizás uno como el de la tarea que no permitía abrir más de 2 ventanas simultaneas, pero carecía del tiempo a estas alturas para reorganizar esto.

2. **Detección de colisiones**

La función de detectar colisiones se encarga de detectar cuándo la pelota entra en contacto con alguna de las paletas durante una partida. Para ello, obtiene las coordenadas de la pelota y de ambas paletas mediante canvas.coords(), lo que permite conocer sus posiciones dentro del área de juego.

Primero, se calcula el radio de la pelota y su centro a partir de sus coordenadas. Después, se estima la posición que tendrá en el siguiente movimiento utilizando las velocidades horizontales (dx) y vertical (dy). Con estos datos, se comprueba si la

pelota coincide verticalmente con la paleta y si está a punto de cruzar alguno de sus bordes.

En el modo fácil contra la inteligencia artificial, se comprueba la colisión con la paleta del jugador, ubicada a la derecha, y con la paleta controlada por la inteligencia artificial, ubicada a la izquierda. Cuando se detecta una colisión, la función devuelve un valor que identifica la paleta involucrada: 1 para la paleta derecha, -1 para la izquierda y 0 cuando no existe ninguna colisión.

Este procedimiento permite detectar los contactos y evitar, en la medida de lo posible, que la pelota atraviese las paletas debido a la velocidad de su movimiento.

3. Mover bola

La función mover_bola() controla el desplazamiento de la pelota durante la partida y utiliza el resultado de detectar_colision() para determinar cuándo debe cambiar su dirección.

Al comenzar, obtiene las coordenadas actuales de la pelota y calcula su radio y centro. Luego, utiliza las velocidades dx y dy para predecir su siguiente posición. Si la pelota se aproxima al borde superior o inferior del área de juego, se invierte la velocidad vertical para producir el rebote.

Después, se llama a detectar_colision() para comprobar si la pelota entra en contacto con alguna paleta. Si se detecta una colisión, la pelota se coloca fuera del borde correspondiente de la paleta y se invierte su velocidad horizontal. De esta manera, cambia de dirección sin quedar superpuesta con la paleta.

La función también comprueba si la pelota sale del área de juego por alguno de los lados. Cuando esto sucede, se actualiza la puntuación del jugador correspondiente y la pelota vuelve a su posición inicial para continuar la partida.

Finalmente, se utiliza juego.after() para programar la siguiente actualización del movimiento después de un intervalo de tiempo. Esto permite mantener el movimiento continuo de la pelota sin detener la ejecución de la interfaz gráfica de Tkinter.

El sistema desarrollado para el modo fácil sirvió como base para implementar las colisiones en las demás modalidades del juego. Se mantuvo la lógica de calcular las coordenadas de la pelota, predecir su siguiente posición, comprobar el contacto con las paletas e invertir la dirección del movimiento cuando corresponde.

En los modos que utilizan dos pelotas, la función mover_bola() recibe como parámetros el objeto de la pelota, sus velocidades y sus coordenadas iniciales. Esto

permite controlar cada pelota de manera independiente y reiniciarla en su propia posición cuando se anota un punto.

En el modo difícil y en las modalidades que incluyen obstáculos, también se incorporan comprobaciones adicionales para detectar el contacto con dichos elementos y modificar la dirección de la pelota.

En conjunto, estas funciones permiten controlar el movimiento, los rebotes y la puntuación, y constituyen una parte fundamental del funcionamiento del juego.

4. Movimiento de paletas

Para controlar el movimiento de la **paleta del jugador**, se utiliza una lista llamada movimiento, que almacena la dirección en la que debe desplazarse. El valor -1 indica que la paleta debe subir, el valor 1 indica que debe bajar y el valor 0 indica que debe permanecer quieta.

La función mover_paleta() se encarga de actualizar la posición de la paleta en la pantalla. Primero, comprueba si la partida continúa activa y obtiene las coordenadas de la paleta mediante canvas.coords(). Después, verifica el valor almacenado en movimiento para determinar si debe desplazarse hacia arriba o hacia abajo.

El desplazamiento se realiza mediante canvas.move(), que permite mover la paleta 10 píxeles en el eje vertical. Antes de realizar el movimiento, se comprueba que la paleta no sobrepase los límites superior e inferior del área de juego, evitando que desaparezca de la pantalla.

Para conseguir un movimiento continuo, se utiliza juego.after(10, mover_paleta), que programa una nueva ejecución de la función después de 10 milisegundos. De esta manera, la posición se actualiza repetidamente mientras se mantiene presionada una tecla.

El control de la paleta se divide en dos funciones: **presionar_tecla() y soltar_tecla().** La función presionar_tecla(event) detecta cuándo el jugador presiona una tecla. Si se presiona la tecla W, el valor de movimiento cambia a -1, haciendo que la paleta suba. Si se presiona la tecla S, el valor cambia a 1, haciendo que la paleta baje.

Por otro lado, la función soltar_tecla(event) detecta cuándo se libera cualquiera de estas dos teclas y restablece el valor de movimiento a 0. Esto permite que la paleta se detenga cuando el jugador deja de presionar la tecla.Para vincular estas funciones con el teclado, se utiliza el método juego.bind(), que relaciona los eventos de presionar y soltar las teclas con sus respectivas funciones. Finalmente,

juego.focus_set() establece el enfoque en la ventana del juego para facilitar la recepción de los eventos del teclado.

La paleta de la inteligencia artificial se controla mediante un algoritmo sencillo de seguimiento de la pelota. A diferencia de la paleta del jugador, esta no depende de las teclas, sino que ajusta su posición automáticamente según la ubicación de la pelota en el eje vertical.

La función mover_ia() obtiene las coordenadas de la pelota y de la paleta de la inteligencia artificial mediante canvas.coords(). Posteriormente, compara la coordenada superior de la pelota con la posición vertical de la paleta para determinar si debe subir o bajar.

Si la pelota se encuentra por encima de la zona central de la paleta, la inteligencia artificial se desplaza hacia arriba. Si la pelota se encuentra por debajo, se desplaza hacia abajo. En ambos casos, se comprueba que la paleta no supere los límites de la ventana.

El desplazamiento de la inteligencia artificial es de 4 píxeles por actualización, utilizando canvas.move(). Además, la función se vuelve a ejecutar cada 30 milisegundos mediante juego.after(30, mover_ia), lo que permite revisar continuamente la posición de la pelota y ajustar el movimiento de la paleta.

Este algoritmo permite que la inteligencia artificial siga la trayectoria vertical de la pelota de forma autónoma. Como su velocidad de desplazamiento es menor que la del jugador, el comportamiento no garantiza que siempre alcance la pelota, lo que contribuye a establecer un nivel de dificultad básico.

Ambos sistemas de movimiento comprueban si la partida continúa activa antes de realizar sus actualizaciones. Cuando juego_activo[0] pasa a ser False, las funciones dejan de programar nuevos movimientos.

De esta manera, el movimiento manual del jugador y el seguimiento automático de la inteligencia artificial funcionan simultáneamente durante la partida. El uso de juego.after() permite actualizar ambas paletas de forma periódica sin bloquear la interfaz gráfica de Tkinter.

Esta estructura permitió en el jugador 2 combinar elementos de la paleta ia y del jugador 1 con la detección de teclas y las ubicaciones de la paleta.

5. **Ventana de configuración**

También se incorpora una imagen de fondo utilizando las bibliotecas Pillow, mediante las clases Image e ImageTk. La imagen se abre, se ajusta al tamaño de

El botón de efectos de sonido permite activar o desactivar los sonidos de la interfaz y del juego, según la configuración implementada en el módulo sonidos.py.

La función interna cambiar_efectos() consulta el estado actual almacenado en sonidos.efectos_activos[0] y utiliza el operador lógico not para invertirlo. Después, llama a sonidos.activar_efectos(estado_actual) para actualizar el estado del sistema de sonido.

Según el nuevo estado, el texto del botón cambia entre «Efectos: Activados» y «Efectos: Desactivados». Esto permite que el usuario identifique visualmente la configuración seleccionada.

El estado inicial del botón se obtiene directamente del módulo sonidos, de modo que el texto mostrado corresponda con la configuración actual. Además, el comando del botón ejecuta la reproducción del efecto de sonido y el cambio de estado.

Para modificar el volumen de la música, se utiliza un control deslizante creado con tk.Scale(). Este control permite seleccionar un valor entre 0 y 100, donde 0 representa el volumen mínimo y 100 el máximo.

El valor seleccionado se almacena en una variable de Tkinter de tipo DoubleVar, inicializada a partir de musica.volumen_musica[0]. De esta manera, la barra comienza mostrando el volumen registrado en el módulo de música.

La función ajustar_volumen(valor) recibe el valor seleccionado en la barra, lo convierte a un número decimal y lo divide entre 100 para obtener un valor comprendido entre 0.0 y 1.0. Posteriormente, llama a musica.cambiar_volumen() para aplicar el nuevo nivel de volumen.

Esta conexión permite que el usuario ajuste el volumen desde la interfaz sin necesidad de modificar directamente el código del módulo de música.

El botón «Personalizaciones» permite acceder a una ventana independiente donde se agrupan las opciones para modificar diferentes elementos visuales y sonoros del juego.

Al presionarlo, se reproduce el efecto de sonido de la interfaz y se llama a personalizaciones.abrir_personalizaciones(window), enviando como argumento la ventana de configuración actual.

La función de personalizaciones puede utilizar esa referencia para cerrar la ventana anterior antes de abrir la nueva, según la implementación del módulo personalizaciones.py. Esto permite mantener una navegación organizada entre las diferentes ventanas del programa.

Las opciones de personalización se gestionan desde su propio módulo, lo que facilita separar la interfaz principal de configuración de las funciones encargadas de cambiar los fondos, la música y los colores de los elementos del juego.

6. Personalizaciones

El sistema permite seleccionar entre cinco imágenes diferentes para personalizar el fondo de la partida. Los nombres de los archivos se almacenan en la lista fondos_disponibles.

Para mostrar las opciones, se utiliza un componente tk.OptionMenu, vinculado a la variable fondo_elegido, de tipo tk.StringVar. Esta variable almacena el nombre del fondo seleccionado por el usuario.

La función actualizar_vista_previa(nombre_fondo) permite mostrar una vista previa de la imagen elegida. Para ello, abre el archivo correspondiente, cambia sus dimensiones a 250 × 140 píxeles, crea una imagen compatible con Tkinter y la coloca en la etiqueta vista_previa.

Por su parte, la función seleccionar_fondo(nombre_fondo) actualiza la opción seleccionada, refresca la vista previa y guarda el nombre del archivo en personalizacion_datos.fondo_seleccionado.

Este procedimiento permite que el usuario visualice los diferentes fondos antes de utilizarlos. El módulo personalizacion_datos funciona como un lugar compartido para almacenar la selección, de modo que otros módulos del programa puedan consultarla.

La sección de música permite elegir entre dos canciones disponibles, cuyos nombres se obtienen de las variables musica.tema_1 y musica.tema_2. La variable cancion_elegida, creada con tk.StringVar, almacena la opción seleccionada. El menú desplegable selector_musica presenta las canciones disponibles y permite elegir una de ellas.

Cuando el usuario selecciona una canción, se ejecuta la función seleccionar_cancion(nombre_cancion). Esta función actualiza el valor de cancion_elegida y guarda la elección en personalizacion_datos.cancion_seleccionada.

De esta forma, la selección musical queda disponible para que el módulo encargado de la reproducción pueda consultarla y utilizarla cuando corresponda.

La función interna abrir_colores() se encarga de abrir una ventana independiente para personalizar los colores de los elementos de la partida.

Primero, cierra la ventana de personalizaciones y crea otra ventana secundaria titulada «Colores de la partida». Al igual que las ventanas anteriores, utiliza un fondo gráfico, dimensiones fijas y un título.

Dentro de esta ventana se presentan tres botones que permiten modificar el color de la paleta del jugador 1, la paleta del jugador 2 o la inteligencia artificial, y todas las pelotas. La función cambiar_color(tipo) permite seleccionar el elemento cuyo color se desea modificar.

Mediante una condición, se identifica si el usuario desea cambiar el color del jugador 1, del jugador 2 o de las pelotas. Después, se utiliza colorchooser.askcolor() para abrir el selector de colores de Tkinter.

Si el usuario elige un color válido, se guarda el código hexadecimal correspondiente en la variable adecuada del módulo colores.py:

- colores.color_jugador_1: almacena el color de la paleta del jugador 1.
- colores.color_jugador_2: almacena el color de la paleta del jugador 2 o de la inteligencia artificial.
- colores.color_pelota: almacena el color de las pelotas.

También se actualiza el color de fondo del botón correspondiente mediante config(bg=resultado[1]). Esto permite visualizar inmediatamente el color seleccionado en la interfaz.

Si el usuario cancela el selector, no se modifica ninguna de las variables de color.

El botón «Colores de la partida» permite abrir la ventana de selección de colores. Su comando reproduce primero el efecto de sonido de la interfaz mediante sonidos.reproducir_sonido() y después ejecuta la función abrir_colores().

Esta organización permite separar las opciones de selección de fondos y música de las opciones de colores, evitando que todos los controles deban mostrarse en una sola ventana.

El archivo utiliza diferentes módulos para mantener separadas las responsabilidades del programa:

- sonidos.py: reproduce los efectos de sonido de la interfaz.
- musica.py: contiene las referencias a las canciones disponibles y las funciones relacionadas con la reproducción musical.
- colores.py: almacena l

Para finalizar se hace constar que el uso de IA en este proyecto viene referenciada en un apartado propio del documento, se implementó para soluciones de errores de indentación investigación de propias funciones de tkinter útiles para el proyecto como las librerías recomendadas para ciertas partes del proyecto y el manejo adecuado de ventanas, así como evaluaciones de diseño en las etiquetas tipos de letra y el mismo código rgb hexadecimal para los colores y la paleta de colores del modo personalizado.

Referencias

OpenAI. (2026). *ChatGPT* [Modelo de lenguaje grande]. https://chatgpt.com/

Python Software Foundation. (s. f.). *Graphical user interfaces with Tk*. Python 3.14 documentation. https://docs.python.org/3/library/tk.html

Python Software Foundation. (s. f.). *Tkinter — Python interface to Tcl/Tk*. Python 3.14 documentation. https://docs.python.org/3/library/tkinter.html

Anexo

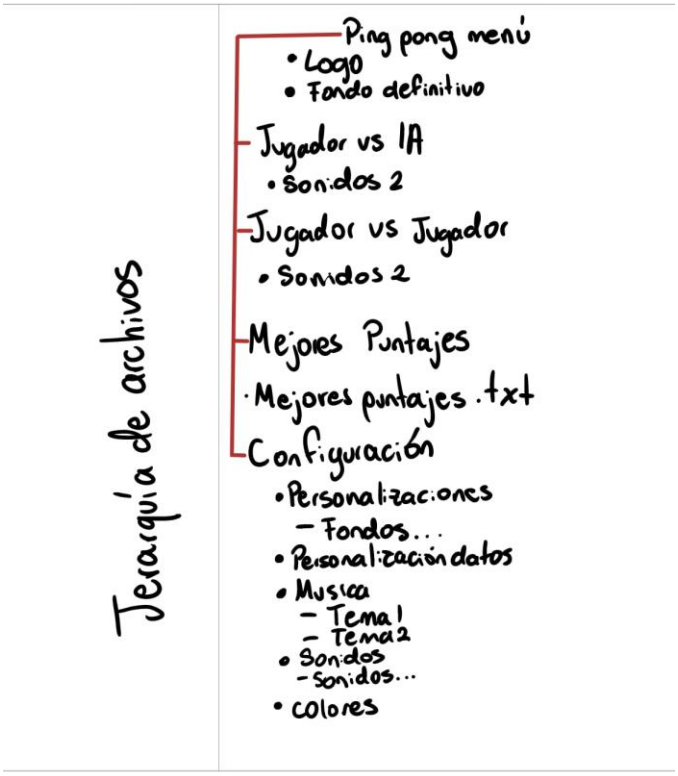


Diagrama de jerarquía del proyecto Ping Pong